AVIGNON UNIVERSITÉ

Amiibo Quiz – Application mobile Android

Mohamed Adnane

2025

UFR
SCIENCES
TECHNOLOGIES
SANTÉ

AVIGNON UNIVERSITÉ
CERI

CENTRE
D'ENSEIGNEMENT
ET DE RECHERCHE
EN INFORMATIQUE
ceri.univ-avignon.fr

Sommaire

Titre 1

Sommaire 2

1 Modélisation UML 3
- 1.1 Diagramme de cas d'utilisation . . . 3
- 1.2 Diagramme de classes . . . 4

2 Ergonomie et utilisabilité 5
- 2.1 Évolution de l'ergonomie : de la version initiale à la version finale . . . 5
- 2.2 Composants Android utilisés . . . 9

3 Implémentation technique 9
- 3.1 Appel API avec Retrofit . . . 9
- 3.2 Base de données Realm . . . 10
- 3.3 Génération des questions de quiz . . . 10

4 Problèmes rencontrés et solutions détaillées 10
- 4.1 Problème SSL Handshake avec l'API . . . 10
- 4.2 Incompatibilité Realm Kotlin SDK . . . 11
- 4.3 Intégration corrigée : Musique de fond persistante . . . 11
- 4.4 Non-conservation de la sélection à MainActivity . . . 11
- 4.5 Perte de favoris après changement de GameSeries . . . 11

5 Tests, feedback et améliorations 12
- 5.1 Tests réalisés . . . 12

6 Retour critique 13

7 Fonctionnalités demandées et réalisées 14

Introduction

Dans le cadre de l'UE « Application Mobile et Ergonomie » dispensée en Licence 3 Informatique au CERI d'Avignon, nous avons été amenés à concevoir une application Android complète. Le projet proposé, que nous avons baptisé **Amiibo Quiz**, a pour but de permettre aux utilisateurs de tester leurs connaissances sur les personnages de l'univers Amiibo — un ensemble de figurines connectées de Nintendo.

J'ai profité de la période où les autres étudiants étaient en alternance pour avancer pleinement sur ce projet. Cela m'a permis de me concentrer sur l'ensemble des étapes (modélisation, développement, tests), tout en respectant les contraintes techniques et ergonomiques du sujet.

Ce projet nous a permis de mettre en pratique plusieurs compétences-clés abordées durant le semestre :

- **Modélisation UML** : conception préalable de l'application à l'aide de diagrammes de cas d'utilisation, de classes et de séquence. Cette phase a joué un rôle crucial pour structurer le projet avant l'implémentation.
- **Programmation mobile Android** : création d'interfaces adaptées via ViewBinding, affichage dynamique avec RecyclerView, navigation entre activités grâce aux intents, et gestion de ressources multimédias (musique de fond).
- **Ergonomie** : intégration de principes abordés en cours et en TP. Nous avons notamment mis en œuvre des interactions directes avec l'utilisateur comme les Toasts pour les messages rapides (succès, erreurs), les AlertDialogs pour les avertissements importants (erreurs API ou confirmation), ainsi qu'une interface visuelle claire et épurée. Tous ces éléments visent à améliorer l'expérience utilisateur selon les critères d'utilisabilité étudiés durant le semestre.
- **Persistance des données** : enregistrement local des favoris et des sélections de l'utilisateur à l'aide du SDK Realm Java, conformément aux TPs réalisés.
- **Communication avec une API REST** : intégration de l'API Amiibo via Retrofit pour charger les personnages dynamiquement, en fonction des choix de l'utilisateur.

1 Modélisation UML

1.1 Diagramme de cas d'utilisation

L'utilisateur est l'acteur principal de l'application. Il peut effectuer différentes actions représentées par les cas d'utilisation suivants :

- Lancer l'application
- Consulter la liste des séries Amiibo
- Sélectionner une ou plusieurs séries
- Valider la sélection et récupérer les Amiibo
- Lancer le quiz
- Répondre à une question
- Ajouter un personnage aux favoris
- Consulter la liste des favoris
- Écouter une musique de fond dans le quiz
- Quitter ou revenir à l'accueil

numéroimage 1 : Diagramme de cas d'utilisation UML

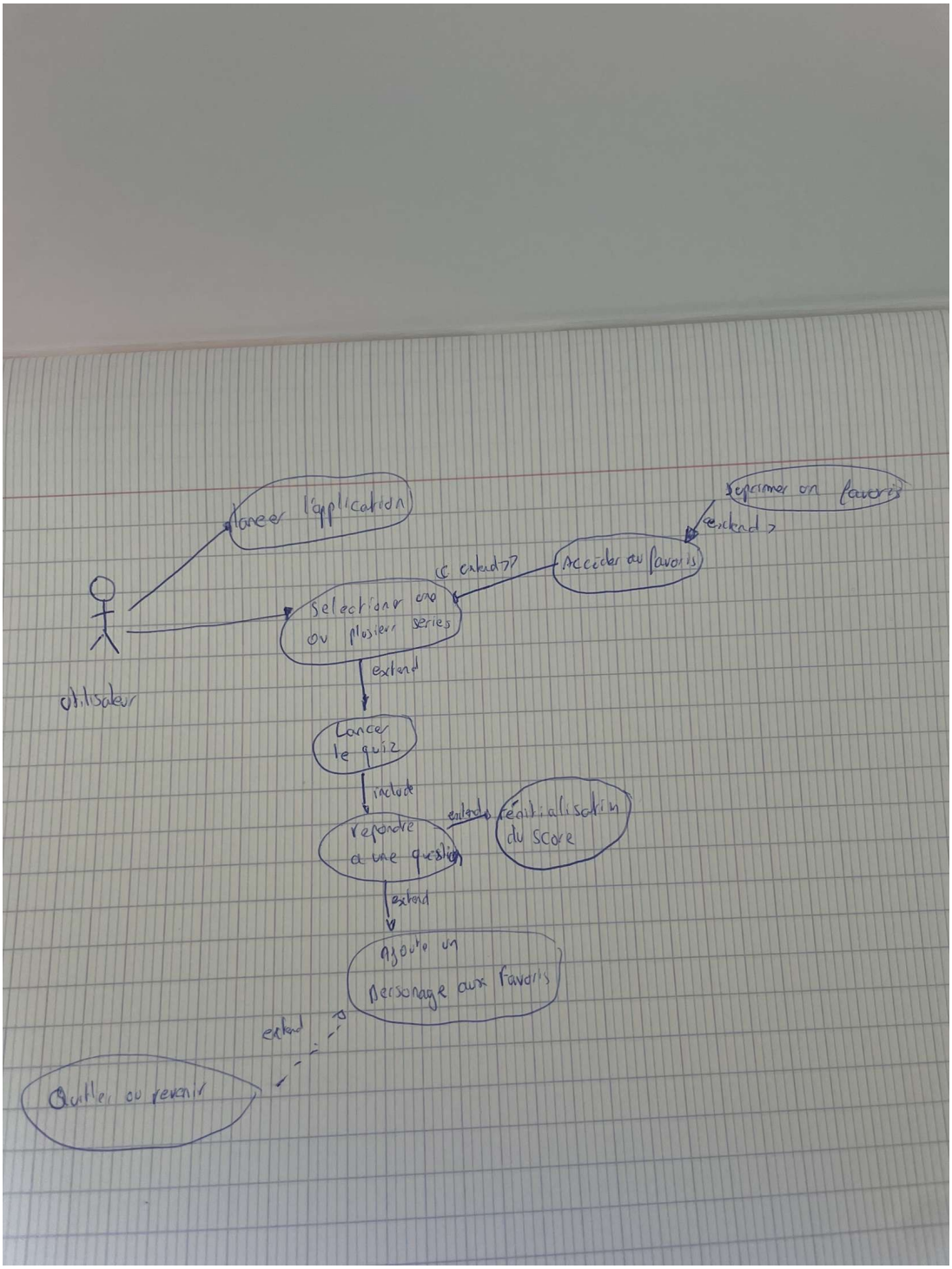


Figure 1. Diagramme de cas d'utilisation UML

1.2 Diagramme de classes

Le diagramme de classes suivant présente les principales entités manipulées par l'application :

- `Amiibo` : contient les attributs `name`, `image`, `tail`, `amiiboSeries`.
- `RealmAmiibo` : classe persistée qui hérite de `RealmObject` et utilise `@PrimaryKey tail`.
- `AmiiboQuestion` : structure pour le quiz contenant l'image, les options et la bonne réponse.
- `MainActivity` : responsable de la gestion des GameSeries.

- `GameActivity` : implémente la logique du quiz.
- `FavorisActivity` : affiche les favoris stockés dans Realm.

numéroimage 2 : Diagramme de classes UML

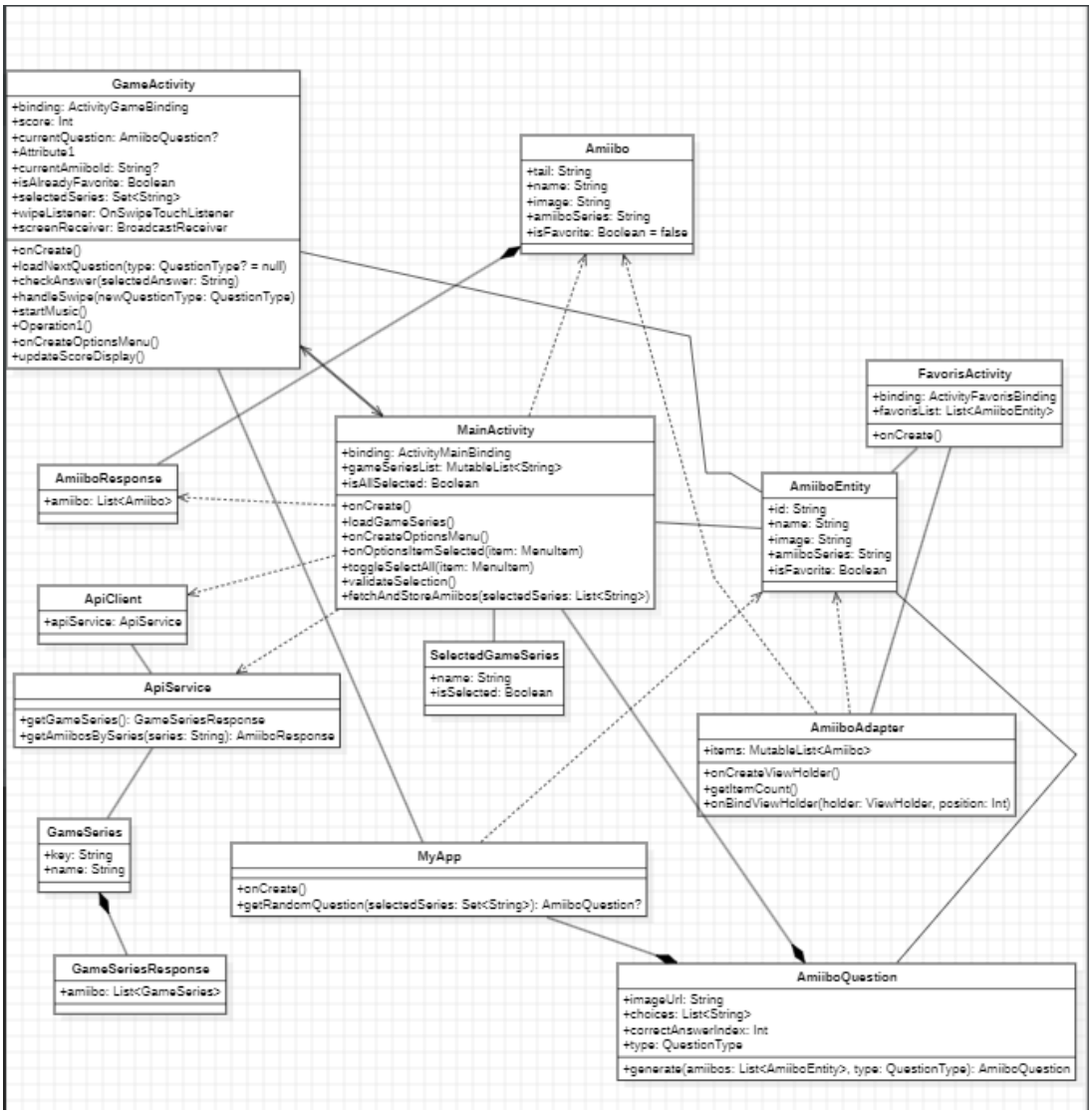


Figure 2. Diagramme de classes UML

2 Ergonomie et utilisabilité

2.1 Évolution de l'ergonomie : de la version initiale à la version finale

Lors de la première phase du projet, l'application Amiibo Quiz disposait d'une ergonomie très basique. Mon objectif initial était de faire fonctionner les fonctionnalités principales avant de me concentrer sur l'aspect visuel ou l'expérience utilisateur. Cela signifie que les premières interfaces étaient avant tout fonctionnelles, sans structure visuelle claire ni éléments interactifs guidant efficacement l'utilisateur. Par exemple, dans la toute première version de MainActivity, les séries Amiibo étaient affichées sous forme brute, sans interactivité particulière. Au fur et à mesure de l'avancement, j'ai progressivement intégré certains choix ergonomiques simples mais efficaces : l'utilisation d'une couleur violette pour attirer l'attention et différencier la barre d'action, la position du bouton de validation en bas à droite afin de

faciliter son accès rapide avec le pouce (en accord avec les bonnes pratiques sur Android), le placement des cases à cocher à droite des noms pour simplifier la sélection en séparant clairement l'information (à gauche) de l'interaction (à droite), ainsi que l'ajout d'un bouton « Favoris » en haut à droite pour un accès immédiat et intuitif aux Amiibos préférés. Ces choix simples ont permis une nette amélioration de l'expérience utilisateur globale

numéroimage 3 : Interface ergonomique améliorée (MainActivity)

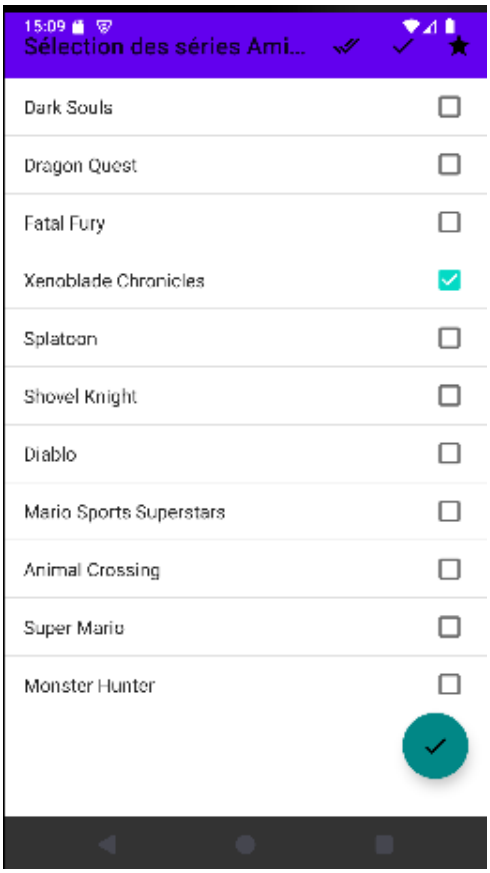


Figure 3. Interface finale de sélection des séries Amiibo

Ensuite, le passage à `GameActivity` marquait un tournant. Dans la première implémentation, les personnages Amiibo étaient à peine identifiables : pas d'images, simple nom affiché dans une `TextView`, aucune interaction.

Dans la version finale, chaque personnage est affiché sous forme de carte (`CardView`), incluant l'image (chargée avec `Picasso`), le nom, et surtout un bouton "Ajouter aux favoris" représenté par une icône en forme d'étoile.

numéroimage 4 : Bouton « Ajouter aux favoris » dans GameActivity

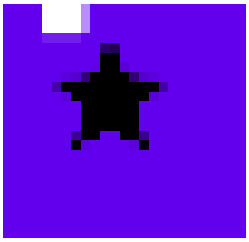


Figure 4. Carte Amiibo avec bouton étoile

Le choix d'une étoile comme symbole de favori est volontaire : c'est une convention d'interface largement répandue sur les plateformes mobiles et web. Elle est immédiatement compréhensible par l'utilisateur, et son aspect visuel attire naturellement l'œil. Cela contribue à l'ergonomie et à la simplicité d'usage de l'application.

L'un des points clés que j'ai amélioré est le **feedback utilisateur**.

Initialement, cliquer sur un bouton n'affichait rien. L'utilisateur ne savait pas si l'action avait fonctionné, ce qui nuisait à la clarté et à l'expérience globale.

J'ai intégré des `Toasts` pour confirmer l'ajout d'un favori, ou pour signaler une erreur réseau (par exemple lorsque l'utilisateur est hors-ligne). Ce retour visuel rapide permet de rassurer l'utilisateur sur le bon déroulement de son action.

numéroimage 5 : Toast de confirmation ou d'erreur réseau

Figure 5. Exemple de Toast après ajout aux favoris

Pour améliorer la robustesse, j'ai aussi ajouté des `AlertDialogs`.

Par exemple, si l'utilisateur clique sur le bouton "Valider" sans avoir sélectionné de série, un message d'alerte s'affiche pour lui rappeler qu'au moins une sélection est requise. Ce type de message d'erreur n'existait pas dans les premières versions, et son ajout contribue à rendre l'application plus intuitive et plus tolérante aux erreurs.

numéroimage 6 : AlertDialog avertissant en cas de validation sans sélection

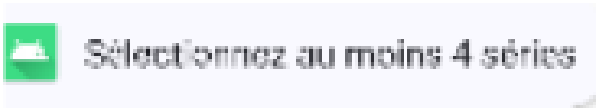


Figure 6. AlertDialog : aucune série sélectionnée

Enfin, j'ai conçu une FavorisActivity claire, avec un fond blanc épuré. Les personnages favoris y sont affichés sous forme de liste, et gérés via Realm pour garantir la persistance des données. Ce nouvel écran améliore considérablement l'expérience utilisateur, car il permet de retrouver facilement les favoris ajoutés, même après avoir quitté l'application ou entre différentes sessions. Chaque carte affiche le personnage Amiibo avec son image et son nom, ainsi qu'un bouton placé au centre inférieur de la carte permettant de retirer facilement un personnage des favoris. Ce positionnement centralisé permet une interaction rapide et évidente, sans surcharger visuellement la carte. Un bouton dans le menu, placé en haut à gauche, permet également de revenir à la MainActivity, offrant ainsi une navigation fluide et intuitive dans l'application.

numéroimage 7 : Interface de la FavorisActivity

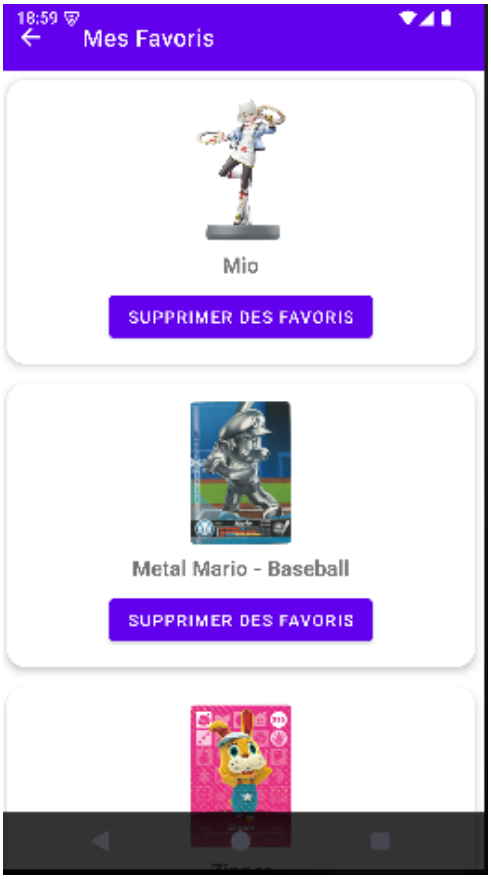


Figure 7. Affichage des personnages favoris via Realm

Pour cette partie du quiz Amiibo, j'ai volontairement fait plusieurs choix ergonomiques afin d'optimiser l'expérience utilisateur. J'ai placé le score en haut à droite de manière discrète mais toujours visible, afin que l'utilisateur puisse suivre sa progression sans que cela prenne trop de place sur l'écran. J'ai également ajouté une icône étoile dans la barre d'action pour permettre l'ajout rapide d'un Amiibo aux favoris, en suivant une logique d'interface familière et cohérente avec les standards Android.

Les questions peuvent varier en fonction du type , mais j'ai conservé une structure simple et claire pour toutes : image centrale, réponses bien espacées au milieu de l'écran, et interactions faciles au toucher. J'ai aussi choisi d'ajouter un bouton de réinitialisation du score, placé à côté du score en haut de l'écran pour être directement visible et accessible, tout en évitant d'encombrer le reste de l'interface.Tous ces choix ont été faits dans un souci de simplicité, d'intuitivité et de cohérence avec les principes d'ergonomie mobile.

Musique de fond et limitations

J'ai aussi ajouté une **musique de fond** dans le quiz (`GameActivity`). Elle démarre automatiquement et reste active tant que l'utilisateur ne quitte pas l'activité. Ce n'est pas seulement un effet sonore, mais un élément immersif de l'application qui accompagne l'utilisateur pendant toute la durée du quiz.

Par manque de temps, certaines idées que j'aurais aimé implémenter n'ont pas pu être finalisées. Notamment, j'aurais souhaité :

- Ajouter un système d'historique permettant à l'utilisateur de revoir ses réponses passées.
- Intégrer un chronomètre limitant le temps de réponse pour chaque question.

Ces fonctionnalités auraient apporté une dimension plus ludique et challengeante à

l'application, mais leur mise en œuvre aurait nécessité une gestion plus poussée des états et du temps que je n'ai pas pu approfondir dans le délai imparti.

2.2 Composants Android utilisés

Dans le cadre de ce projet, plusieurs composants Android ont été intégrés de manière cohérente afin de répondre à la fois aux exigences fonctionnelles et aux principes ergonomiques abordés dans le module.

L'ensemble des choix effectués s'est appuyé sur les acquis des TPs réalisés durant le semestre, notamment les projets sur :

- La gestion de base de données (TP5/TP6 - X-Men)
- Les appels réseau (TP Retrofit)
- Les interactions utilisateur (TP RecyclerView et favoris)

Le cœur de l'affichage dynamique repose sur l'utilisation de `RecyclerView`, couplé à `ViewBinding`, ce qui permet une gestion fluide et typée des vues. J'ai réutilisé la structure mise en place dans le TP des X-Men, en l'adaptant pour afficher les Amiibo dans `GameActivity`, via une carte personnalisée (image + nom + bouton de favori).

Pour le chargement des images, j'ai utilisé la bibliothèque `Picasso`, comme dans le TP X-Men. Elle permet un rendu rapide et sans blocage du thread principal.

La persistance locale a été assurée avec `Realm`, configuré exactement comme dans le TP6 :

- Classe `Application` avec initialisation
- Plugin dans `build.gradle.kts`
- Modèles `RealmObject` avec `@PrimaryKey`

Pour la navigation entre les écrans, j'ai utilisé des `Intent explicites`, comme dans le TP des quiz. Cette méthode permet un passage clair et sécurisé des données entre activités (`MainActivity`, `GameActivity`, `FavorisActivity`).

Enfin, les `Toast.makeText(...)` ont été intégrés aux points stratégiques :

- Confirmation d'ajout aux favoris
- Message d'erreur réseau (utilisé dans TP Retrofit)

Tous ces choix sont directement issus des TPs, et non de ressources modernes externes. Cela assure un alignement total avec les objectifs pédagogiques du module, tout en maintenant une architecture solide et maintenable.

3 Implémentation technique

3.1 Appel API avec Retrofit

L'application utilise Retrofit pour récupérer la liste complète des personnages via l'API Amiibo. Une fois les données récupérées, elles sont filtrées selon les `GameSeries` sélectionnées, puis insérées localement dans Realm :

```
val selectedSeries = selectedGameSeries.joinToString("|") { it }
retrofitService.getAmiibos().enqueue(object : Callback<AmiiboResponse> {
    override fun onResponse(...) {
        val filtered = allAmiibos.filter { it.amiiboSeries in selectedSeries }
        realm.executeTransaction {
            it.insertOrUpdate(filtered.map { it.toRealm() })
        }
    }
})
```

3.2 Base de données Realm

La base de données locale est assurée par `Realm`, utilisée exactement comme dans les TPs. Voici les éléments principaux de l'implémentation :

- Classe `RealmAmiibo` avec l'annotation `@PrimaryKey tail`
- Initialisation de Realm dans la classe `MyApplication`
- Utilisation dans l'application :
  - **Ajout** aux favoris via `realm.executeTransaction`
  - **Suppression** (optionnelle)
  - **Lecture** via `realm.where<RealmAmiibo>()...`

```
verride fun onCreate() {
    super.onCreate()

    Realm.init( context: this)
    val config = RealmConfiguration.Builder()
        .allowWritesOnUiThread(true)
        .name( filename: "amiibo.realm")
        .schemaVersion(1)

        .build()

    Realm.setDefaultConfiguration(config)
}
```

Ajout dans Realm

```
ackage fr.ceri.amiboprojetfinal

mport io.realm.RealmObject
mport io.realm.annotations.PrimaryKey

pen class RealmAmiibo(
    @PrimaryKey var tail: String = "",
    var name: String = "",
    var image: String = "",
    var amiiboSeries: String = "",
    var isFavorite: Boolean = false
) : RealmObject()
```

Lecture des favoris

Figure 8. Gestion des favoris avec Realm

3.3 Génération des questions de quiz

Chaque question du quiz est générée dynamiquement à partir de la liste d'Amiibo sélectionnée. Voici la fonction utilisée :

```
fun generateQuestion(): AmiiboQuestion {
    val current = amiibos.random()
    val options = amiibos.shuffled().take(3).map { it.name }.toMutableList()
    options.add(current.name)
    options.shuffle()
    return AmiiboQuestion(current.image, options, current.name)
}
```

La logique est la suivante :

- Sélection aléatoire d'un Amiibo comme bonne réponse
- Génération de trois noms d'Amiibo au hasard comme distracteurs
- Ajout du bon nom dans les propositions
- Mélange final des options pour brouiller la position

4 Problèmes rencontrés et solutions détaillées

4.1 Problème SSL Handshake avec l'API

Ce que je voulais faire : appeler l'API en HTTPS via Retrofit, comme en TP.
Problème : `SSLHandshakeException` bloquant l'accès à l'API sur l'émulateur Android.

Tests : changement de device physique (parfois OK), tentative de désactivation de la sécurité (non conforme, abandonnée).
Solution retenue : ajout du certificat SSL manquant dans Android Studio.
Pourquoi : méthode propre, conforme aux TPs, compatible avec Android et sécurisée.

4.2 Incompatibilité Realm Kotlin SDK

Ce que je voulais faire : utiliser la version Kotlin moderne de Realm.
Problème : conflit avec ViewBinding, plugins non reconnus, incompatibilité avec `build.gradle.kts`.
Tests : migration vers Groovy, remplacement par Room (solution trop complexe dans le contexte).
Solution retenue : retour au SDK Java de Realm, comme dans le TP6 (X-Men).
Pourquoi : stable, bien documenté, compatible avec les outils utilisés en TP.

4.3 Intégration corrigée : Musique de fond persistante

Ce que je voulais faire : ajouter une musique de fond dans `GameActivity` pour renforcer l'immersion.
Problème initial : la musique redémarrait à chaque changement d'activité ou rotation d'écran.
Tests : implémentation d'un `MediaPlayer` local dans `GameActivity`, gestion avec `onPause()`, `onResume()`, puis tentative via un `Service` (solution trop lourde).
Correction finale : gestion complète avec les fonctions `startMusic()`, `pauseMusic()`, `resumeMusic()`, `onDestroy()`, avec `isLooping = true`.
Limite actuelle : sur certains appareils, la musique s'arrête après plusieurs retours arrière – probablement un effet de la destruction de l'activité.

4.4 Non-conservation de la sélection à MainActivity

Ce que je voulais faire : conserver les cases cochées même après un retour depuis `GameActivity`.
Problème : au retour sur `MainActivity`, les cases sont décochées, obligeant l'utilisateur à tout re-sélectionner.
Limite : pas de persistance des sélections dans cette version.

4.5 Perte de favoris après changement de GameSeries

Ce que je voulais faire : permettre de conserver les favoris même après un rechargement de séries.
Problème : lors de chaque nouvelle sélection de GameSeries, tous les Amiibo de Realm étaient supprimés, y compris les favoris.
Tests :
- Désactivation de la suppression globale de Realm
- Séparation logique entre Amiibo "à jouer" et Amiibo "favoris"

Solution retenue : les favoris sont enregistrés dans un objet distinct (`RealmAmiibo`) et filtrés indépendamment. Plus aucune suppression globale de la base Realm.
Résultat : les favoris sont conservés même après changement de GameSeries, et restent consultables depuis l'écran `FavorisActivity`.

5 Tests, feedback et améliorations

5.1 Tests réalisés

Tout au long du développement, j'ai réalisé plusieurs tests manuels pour vérifier la robustesse et l'ergonomie de l'application. Voici les principaux cas testés avec des exemples concrets :

- **Système de score** : vérification du bon fonctionnement du calcul dans le quiz.
  À chaque question :
  - Bonne réponse : `+2 points`
  - Mauvaise réponse : `-2 points`
  - Swipe vers la question suivante sans répondre : `-1 point`

  Chaque action est suivie d'un `Toast` informant l'utilisateur du résultat immédiat.
- **Réinitialisation du score** : testée via le menu dédié dans `GameActivity`. En cliquant sur l'icône correspondante, le score est bien remis à zéro et l'affichage dans la barre d'outils est mis à jour instantanément.

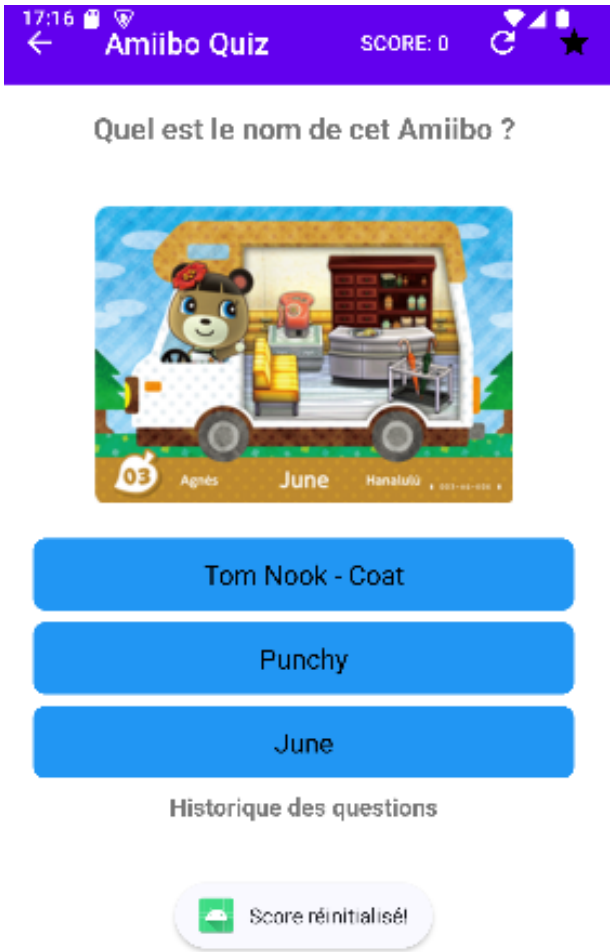
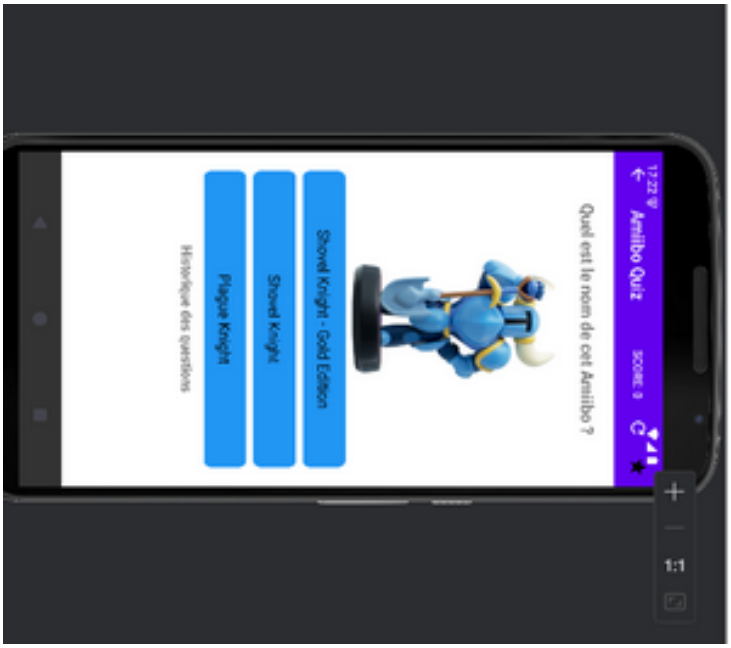


Figure 9. Affichage des personnages favoris via Realm

- **Ajout aux favoris** : chaque clic sur le bouton "Ajouter aux favoris" dans `GameActivity` est bien enregistré dans Realm. J'ai vérifié que les données persistent même après fermeture et redémarrage de l'application.

- **Filtrage des séries sélectionnées** : après avoir coché certaines séries dans `MainActivity`, seuls les Amiibo correspondant à ces séries sont utilisés pour générer les questions du quiz. Ce filtrage a été validé par observation directe des résultats.
- **Comportement ergonomique** : tous les boutons sont bien espacés, accessibles, et visibles même sur petits écrans. Chaque action déclenche un retour utilisateur immédiat (`Toast` ou `AlertDialog`), ce qui renforce la fluidité globale de l'application.
- **Blocage du mode paysage** : pendant l'utilisation de `GameActivity`, j'ai testé la rotation de l'écran pour vérifier que le mode paysage était bien désactivé. Le comportement est conforme : l'écran reste verrouillé en mode portrait comme demandé dans le sujet.



- **Comportement au retour d'activité** : lorsque l'utilisateur quitte `GameActivity` (ou met l'application en arrière-plan), la musique est automatiquement mise en pause. Elle reprend correctement à la reprise de l'activité grâce à la gestion du cycle de vie avec `onPause()` et `onResume()`.

6 Retour critique

Points positifs :

L'application finale est fonctionnelle, fluide et relativement complète. Les transitions entre écrans sont stables, l'intégration de l'API fonctionne sans crash, et les données des favoris sont bien persistées dans Realm. Les éléments d'ergonomie apportés (feedback utilisateur, AlertDialogs, musique de fond) renforcent nettement l'expérience utilisateur.

Points à améliorer :

- **Système de score** : le quiz n'enregistre pas encore le nombre total de bonnes réponses ni le score final. Une amélioration serait d'afficher un Toast final ou une activité "résultat".
- **Filtrage des favoris par série** : actuellement, tous les favoris sont affichés ensemble. Un filtrage par `amiiboSeries` ou une recherche aurait amélioré la lisibilité.
- **Tests unitaires** : aucun test JUnit automatisé n'a été intégré. Exemple de test possible :

```
@Test
fun testBonneReponseIncluse() {
    val question = generateQuestion()
    assertTrue(question.options.contains(question.bonneReponse))
```

```
}
```

- **Historique ou score final** : une activité pour afficher les réponses données à la fin du quiz avait été commencée, mais pas finalisée. Cela aurait enrichi le suivi de l'utilisateur.
- **Retour au menu principal** : lors du retour à `MainActivity`, les séries précédemment cochées sont perdues. Il aurait été préférable d'enregistrer cette sélection via Realm ou `SharedPreferences`.
- **Icône de l'application** : aucune icône personnalisée n'a été définie dans le manifeste. Cela nuirait à la présentation d'une version publiée sur le Play Store.
- **Amélioration visuelle des séries** : ajouter une icône ou une image à côté du nom de chaque `GameSeries` dans `MainActivity` aurait renforcé l'attrait visuel et la lisibilité.

7 Fonctionnalités demandées et réalisées

- Intégration de l'API `amiiboapi.com` avec Retrofit pour charger dynamiquement les `GameSeries`.
- Affichage des séries sous forme de liste avec Checkbox.
- Menu avec actions : sélection/désélection de toutes les séries et validation via un bouton icône.
- Gestion de la contrainte : sélection minimale de 4 séries avec Toast si non respectée.
- Récupération des Amiibo via API + filtrage par GameSeries + suppression des doublons.
- Stockage dans Realm des Amiibo récupérés.
- Transition vers `GameActivity` pour le quiz.
- Affichage d'un quiz avec 1 image, 3 propositions (via TextViews), gestion du swipe.
- Système de score dynamique : +2 / -2 / -1.
- Réinitialisation du score via menu dans `GameActivity`.
- Intégration d'une musique de fond gérée par le cycle de vie Android.
- Blocage du mode paysage via AndroidManifest.
- Ajout d'Amiibo en favoris via Realm.
- Création de `FavorisActivity` pour leur consultation.
- Feedback utilisateur : Toasts, AlertDialogs.

Fonctionnalités supplémentaires (bonus)

*Fonctionnalités supplémentaires et ressources

Fonctionnalités supplémentaires ajoutées

En plus des fonctionnalités demandées, les éléments suivants ont été intégrés à titre de bonus :

- **Gestion avancée du feedback utilisateur** : utilisation de `Toast.makeText(...)` pour confirmer l'ajout aux favoris, signaler des erreurs API, ou avertir d'une sélection insuffisante de séries.
- **Intégration d'une musique immersive** pendant le quiz, avec contrôle complet du cycle de vie via les méthodes `onPause()`, `onResume()` et `onDestroy()`.
- **Favoris persistés via Realm**, avec possibilité d'ajouter et désormais **de supprimer des personnages favoris** depuis `FavorisActivity`. Cette suppression est immédiate et persistante grâce à l'appel à `realm.executeTransaction{}`.
- **Ajout de boîtes de dialogue interactives** (`AlertDialog`) pour les confirmations ou avertissements, renforçant l'interactivité et la guidabilité de l'application.

- **Tentative d'implémentation d'un historique des réponses** au quiz : une activité dédiée avait été amorcée pour afficher les réponses passées, mais la fonctionnalité n'a pas été finalisée à temps.

Ressources pédagogiques utilisées

- Supports de cours et TPs disponibles sur l'ENT :
  - TP Retrofit
  - TP Realm
  - TP RecyclerView
  - TP X-Men
  - Slides sur l'ergonomie et le cycle de vie Android
- Fichiers fournis en accompagnement :
  - `build.gradle.kts`
  - `network_security_config.xml`
  - Certificats SSL nécessaires à la connexion HTTPS

Documentation et aides en ligne

- **API Amiibo utilisée** :
  https://www.amiiboapi.com
- **Realm SDK Java** :
  https://realm.io/docs/java/latest/
- **Bibliothèque Picasso** :
  https://square.github.io/picasso/
- **Documentation officielle Android** :
  https://developer.android.com
  (Références sur le cycle de vie, les Intents, `MediaPlayer`, `ViewBinding`, etc.)
- **Tutoriels et aides ponctuelles consultées** :
  - Gestion de `SSLHandshakeException` sur Android :
    https://stackoverflow.com/questions/63949333/how-to-fix-sslhandshakeexception-android
  - Conflits de `MediaPlayer` avec le cycle de vie Android :
    https://stackoverflow.com/questions/57781757/how-to-stop-mediaplayer-on-pause-in-android
  - Dynamiser les menus avec `onOptionsItemSelected()` :
    https://discuss.kotlinlang.org/t/android-menu-item-dynamic-update/21795